



S. G. Ganesh

Fail Fast!

When a problem occurs in the software, it should fail immediately, in an easily noticeable way. This 'fail fast' behaviour is desirable, and we'll discuss this important concept in this column.

At first, a 'fail fast' might appear to be a bad practice affecting reliability—why should a system crash (or fail), when it can continue execution? For this, we need to understand that fail fast is very relevant in the context of Heisenbugs.

Consider Bohrbugs, which always crash for a given input, for example, with a null-pointer access. These bugs are easier to test, reproduce and fix. Now, all experienced programmers would have faced situations where the bug that caused the crash just disappears when the software is restarted. No matter how much time and effort is spent to reproduce the problem, the bug eludes us. These bugs are known as Heisenbugs. The effort required to find, fix and test Heisenbugs is an order of magnitude more than the effort required for Bohrbugs. One strategy to avoid Heisenbugs is to turn them into Bohrbugs. How? By anticipating the possible cases in which Heisenbugs can arise, and trying to make them Bohrbugs. Yes, it is not easy, and it is also not always possible, but let us look at a specific example where it is useful.

Concurrent programming is one paradigm where Heisenbugs are common. Our example is a concurrency-related issue in Java. While iterating over a Java collection, we are supposed to modify the collection only through the Iterator methods, such as the *remove()* method. During iteration, if another thread attempts to modify that underlying collection (because of a programming mistake), the underlying collection will get corrupted (i.e., result in an incorrect state). Such an incorrect state can lead to an eventual failure—or if we are fortunate (actually, unfortunate!), the program continues execution without crashing, but gives the wrong results. It is difficult to reproduce and fix these bugs, because such programming mistakes are non-deterministic. In other words, it is a Heisenbug. Fortunately, the Java Iterators try to detect such concurrent modifications, and if found, will throw a *ConcurrentModificationException*, instead of failing late—and that too, silently. In other words, the Java Iterators follow the 'fail fast' approach.

What if a *ConcurrentModificationException* is observed in production software? As the Javadoc for this exception observes, it "...should be used only to detect bugs." In other words, *ConcurrentModificationExceptions* are supposed to be found and fixed during software development, and should not leak to production code. Well, if production software does get this exception, it is certainly a bug in the software, and should be reported to the developer and fixed. At least, we know that there was an attempt for concurrent modification of the underlying data structure, and that's why the software failed (instead of getting wrong results from the software, or failing later with some other symptoms, for which it is not feasible to trace the root cause).

The 'fail-safe' approach is meant for developing robust code. A very good example of writing fail-safe code is using assertions. Unfortunately, there is a lot of unnecessary controversy surrounding the use of asserts. The main criticism is this: the checks are enabled in the development version, and disabled in release versions. However, this criticism is wrong: asserts are never meant to replace the defensive checks that should be put in place in the release version of the software. For example, asserts should not be used to check if the argument passed to a function is null or not. Instead, an if condition should be used to check if the argument is passed correctly, or else an exception, or a premature return, should be performed, as appropriate to the context. However, asserts can be used to do additional checks for assumptions that are made in the code, which are supposed to hold true. For example, a condition that checks that the stack is not empty after a *push* operation is performed on it (i.e., checking for 'invariants').

So, fail fast, be assertive, and you're on the way to developing more robust code. 

By: S G Ganesh

The author works for Siemens (Corporate Technologies), Bangalore. You can reach him at ssganesh@gmail.com.